A Catalogue of Observability and Evaluation Metrics for AI Agents

1st [TODO: Given Name Surname]
[TODO: dept. name of organization]
[TODO: name of organization]
[TODO: City, Country]
[TODO: email address or ORCID]

***Abstract*—[TODO: Rewrite once the structure is settled.] AI agents run many model calls and tool calls per task, and a growing number of tools claim to observe and evaluate them. We ask which observability and evaluation tools are most adopted or most visible, and which agent-execution metrics they provide out of the box. Following an adapted multivocal search over GitHub, the web and the OpenTelemetry conventions, we selected 13 tools and three sets of telemetry conventions beneath them and collected 297 built-in metrics (254 from the tools, 43 from the conventions) from their official documentation, each with a source URL, a verbatim quote and an access date. Merging equivalent metrics yields a taxonomy of 46 unique metrics in 13 categories: 18 observability metrics (whose values are captured from a run) and 28 evaluation metrics (whose values are judgements of agent behaviour or output). A worked example on a retrieval-augmented chatbot, traced with Langfuse and scored with Arize Phoenix and DeepEval, shows 30 of the 46 metrics reported by these three tools in a single run, and shows that a failure the application does not flag as an error is missed by the tracer's error metrics but caught by evaluators.**

***Index Terms*—AI agents, observability, evaluation, LLM, metrics, taxonomy [TODO: confirm keywords]**

I. Introduction

[TODO: Motivation: agents are non-deterministic, multi-step and tool-using, so a final answer alone says little about how a run went. Say why metrics need to be chosen deliberately, and why the vocabulary differs between tools.]

This paper makes three contributions:

1) A taxonomy and a catalogue of 46 observability and evaluation metrics for AI agents, built from 297 built-in metrics documented by the studied tools, each tied to documentation evidence (Section V).
2) The selection of the 13 tools and three sets of telemetry conventions those metrics come from, made with written rules from logged GitHub and Google queries (Section IV).
3) A worked example that shows, on one real application, which of these metrics three of the tools report and where they disagree (Sections II and V-B).

Section II presents the application used as a running example. Section III defines the terms used in the rest of the paper. Section IV explains how the tools were chosen and their metrics collected, and Section V presents the taxonomy and the catalogue, in which each metric is defined and, where the example reported it, illustrated with the value from the example. Sections VI–VIII discuss the results, their limitations and related work. [TODO: Research questions, if the venue expects them (for example: RQ1 which tools; RQ2 which built-in metrics; RQ3 how do metrics overlap across tools).]

II. Motivating example

Throughout the paper we use one real application as a running example. It is small enough to follow in full and it does what an LLM agent does: it calls a model several times, calls tools and composes an answer from what they return.

A. The application

The application is the course chatbot built in the DeepLearning.AI course on Claude Code [1]. It answers questions about four course transcripts. It is a single LLM with two tools: `search_course_content` looks up the passages of the transcripts that match a query, optionally within one course or lesson, and returns them; `get_course_outline` returns the list of lessons of a course. Its input is a user question, with the previous turns of the same conversation as history; its output is a text answer. For each question the model decides whether to call a tool, reads what the tool returns and either calls a tool again or writes the answer. We ran it with `gpt-5-mini` as the model; the model is a setting and can be replaced. The transcripts are stored in a vector database and are not redistributed; the rest of the application, the questions and the recorded results are in the folder `example_rag` of the accompanying repository.

B. The test

The test consisted of twelve scripted questions sent to the chatbot in nine sessions: six questions the transcripts answer (instructor names, lesson contents, a course outline), one question the transcripts do not cover (how to deploy MCP servers on Kubernetes; the chatbot still replies, from the model's own knowledge), one general-knowledge question (the capital of Australia) and one conversation of four turns in which the user states a fact about their project, asks two questions and then complains that the chatbot forgot the fact. The expected answer to each question was taken from the transcripts and written before the run. Every run was traced with Langfuse [2]. The recorded turns were then scored offline with the built-in evaluators of Arize Phoenix [3] and DeepEval [4], with

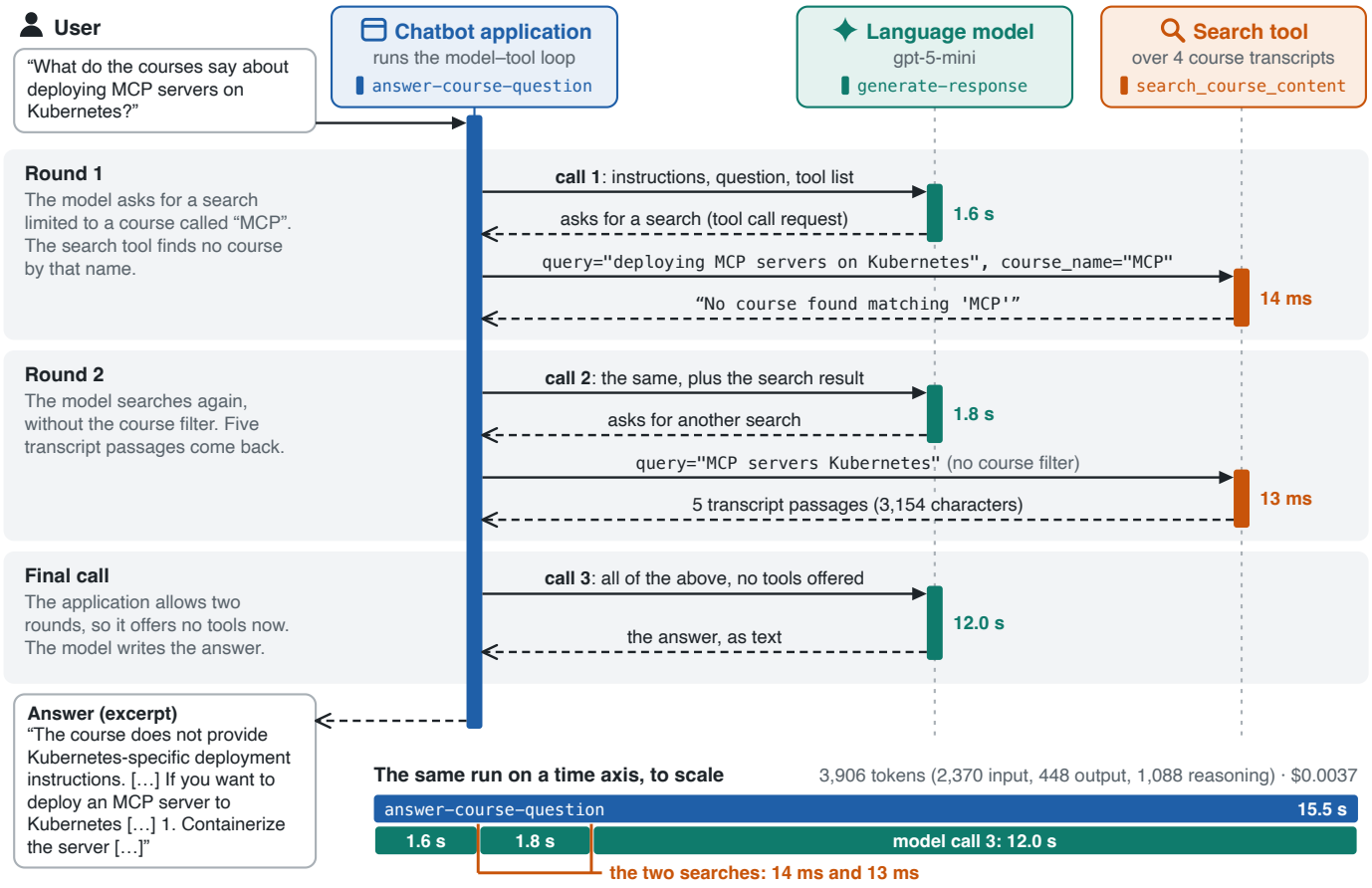


Fig. 1. One run of the example, the question about deploying MCP servers on Kubernetes, from the user's question to the answer (time runs downwards). Solid arrows are requests and dashed arrows are replies. The model never runs a tool itself: it asks for a search, and the application runs it and sends the result back in the next model call. The coloured bars are the operations the tracing tool recorded, with their durations; the name each was recorded under is in monospace at the top of its lane. `answer-course-question` covers the whole run and contains three model calls (`generate-response`) and two searches (`search_course_content`). The strip at the bottom shows the same operations on a time axis.

`gpt-5-mini` again as the judge model. The example illustrates what such tools produce in practice; it does not evaluate them. What the three tools reported is given with the metrics it belongs to (Section V-B).

As an illustration, Fig. 1 shows one of the twelve runs, the question about deploying MCP servers on Kubernetes, as it was recorded. The whole run is one timed operation, `answer-course-question`, which starts when the question arrives and ends when the answer is returned; inside it, the model was called three times and the search tool twice. The model does not run a tool itself: it replies with a request for a tool call, and the application runs the search and sends the result back in the next model call. The first search was restricted to a course named "MCP", which the search tool did not find, and returned no passages; the model searched again without the restriction and five passages came back. The application allows two such rounds, so the third model call offered no tools and the model wrote the answer; that call took 12.0 s of the run's 15.5 s.

III. Background

This section fixes the vocabulary used in the rest of the paper, using the run in Fig. 1 as the reference.

A. Observability

Observability is the ability to understand what a system did from the data it emits; for an LLM agent, this is a record of every model call and tool call made while solving a task. To say what is recorded, we need a unit. A *run* is one execution of the agent, from an input (a user question or a task) to its final output: in Fig. 1, everything from the user's question to the answer. The data recorded for one run is a *trace*. A trace is made of *spans*. A span is one timed operation with a start, an end, a parent and attributes (model name, token counts, tool name, input, output), so spans nest: `answer-course-question` is the parent span of the run, and each model call and each search is a span inside it. A *step* is one action of the agent, a model call or a tool call, and usually corresponds to one span; spans also record other work, such as a retrieval. A *session* groups the runs of one conversation. The tools have their own names for these units; in this paper the

terms above are used for every tool, and "run" always means the whole execution.

With the unit fixed, we can say what is computed from it. For the first two terms we follow OpenTelemetry. There, a metric instrument is defined by a name, a kind, an optional unit and an optional description [5]; a measurement is a data point (a value with attributes) reported through an instrument, and many measurements are combined by aggregation [5], [6]. Accordingly, a *metric* is a named, defined quantity. It states what is measured (a span, a run or a set of runs), the unit or scale, and how results are combined, so that it can be computed again and compared. "LLM call latency" is a metric. A *measurement* is a number obtained by applying such a metric to what was recorded: the third model call in Fig. 1 took 12.0 s, or the median over the twelve runs was 3.0 s. We call a metric an *observability metric* when it is computed from measurements captured during the run (tokens, cost, latency, counts, errors).

Most of the tools studied sit on a shared telemetry layer, which shapes what they can report. OpenTelemetry [7], an open standard for collecting traces, is the base: its conventions for LLM and agent spans fix which attributes a span carries (model name, token counts, tool name) and what they mean. OpenInference [8] and OpenLLMetry [9] are built on it. Both are instrumentation libraries that wrap an LLM or an agent framework and emit spans that follow such conventions; OpenLLMetry states that its own conventions are now part of OpenTelemetry. We call the three *telemetry conventions*, after what we take from them: the names and meanings they give to what a trace contains. Only OpenTelemetry is a standard in the usual sense, a vendor-neutral specification; OpenInference and OpenLLMetry are open-source projects maintained by Arize and Traceloop. The platforms and frameworks studied here sit on top: they store and display the spans and compute metrics from them, or run evaluators over them. Most of them are built on OpenTelemetry or accept its traces (Table I, last column). Two consequences follow. First, the observability metrics are largely the same across tools because they are read from the same span attributes: Phoenix derives cost from the OpenInference token counts, and the OpenTelemetry conventions define the same quantities as metrics (duration of an agent invocation, counts of tool calls and model calls, token usage). Agreement between two tools on tokens, latency or counts is therefore weaker evidence than agreement between independent designs. Second, in the pages we read, the conventions give an attribute for the score an evaluator produced but no list of evaluation metrics. Faithfulness or task completion is defined by each tool, and this is where the vocabularies differ most.

B. Evaluation

Evaluation judges whether what the agent did, and what it answered, was good. Observability records; evaluation needs a judge: a rule, a model or a person. In the example, whether the answer about Kubernetes stayed within what the transcripts say is not in the trace; a judge has to read the answer and the retrieved passages and decide. The tools studied here offer observability, evaluation or both.

The following terms are our own usage, not established vocabulary. A *score* is the number an evaluator assigns to an output or a behaviour, for example a faithfulness of 0.9 given by an LLM judge. We call a metric an *evaluation metric* when it is a score from a judgement. The tools themselves draw a similar line without naming it: Arize Phoenix describes traces as showing "exactly what happened during a run" and, separately, scoring outputs with evaluators [3], and LangSmith lets feedback "score an individual run" [10]. The difference matters in use: a measurement is read from the trace, while a score depends on the judge (its prompt, its model or its reviewer) and can change when the judge changes. Surveys of LLM judges report that small changes in prompt wording or example order can give unstable results, and that the same input can receive different scores [11]; judges also show position, verbosity and self-enhancement bias [12]. Each tool's own metric is a *tool-specific metric*, with its own procedure; a *unique metric* groups tool-specific metrics that measure the same thing (Section V-A), so two tools can report different measurements, or different scores, for one unique metric.

IV. Method

Our goal is a taxonomy and a catalogue of observability and evaluation metrics for AI agents. Rather than proposing metrics, we collect the ones that practitioners can use today: the metrics that observability and evaluation tools ship, as documented by the tools themselves. To do this we (A) selected the most adopted or most visible tools with written rules, (B) read their documentation and recorded every built-in metric that describes how an agent run executes or how good its output is, and (C) grouped equivalent metrics into unique metrics and categories (Section V-A). Every recorded metric carries a documentation URL, a verbatim quote and an access date, and a script checks that each quote appears word for word in a saved copy of its page. The search logs, the evidence tables and the saved pages are in the accompanying repository.

A. Tool selection

Most tools are documented in repositories, vendor pages and blogs rather than in papers, so the search adapts the multivocal approach of Garousi et al. [13]: a search that covers such grey literature alongside papers, with fixed queries, written inclusion and exclusion rules, and a log of every query. We searched two channels on 2026-10-22. On GitHub, 17 queries through the API (10 topics and 7 keywords, 30 repositories each, limited to repositories with at least 1,000 stars, not archived and pushed in the last six months) returned 182 distinct repositories. On Google, six queries (two planned "comparison" queries were dropped) gave 173 results in their first 30 hits, whose titles and snippets named 68 tools. The GitHub queries can be rerun with the same limits; Google results depend on the date and the session, so that channel can be repeated but not reproduced exactly. The OpenTelemetry conventions for generative AI [7] were added as a third entry

by hand, since a specification has no repository or product page that such queries rank.

Candidates were then screened by hand against written rules. A tool was considered if its primary purpose is tracing, monitoring or evaluating LLM applications or agents. An open-source tool also had to meet the star and activity limits above, and a closed-source tool had to have public documentation. We left out gateways and proxies, agent-building frameworks, model-level benchmark harnesses, and tutorials and lists. We also left out general-purpose observability, data, security or ML platforms whose LLM support is only a module, unless the LLM product is distributed on its own. An SDK and its own platform count as one tool. Of the 182 repositories, 145 were removed in this way (68 unrelated, 57 tutorials or lists, 7 gateways, 7 agent-building frameworks, 6 model-level harnesses), which left 37; the 68 tools named by Google were screened with the same rules. Merging the two lists gave 42 candidates.

Two checks followed. First, we read each candidate's own description and kept the 31 that give observability or evaluation of LLM applications or agents as their primary purpose. Second, we kept the 30 whose official documentation explicitly covers agents, tool calls or multi-step traces, evidenced by a quote from a saved copy of the page. Each of the 30 received one *role*: *platform* (an application that stores and shows traces and offers evaluation or monitoring on top of them), *framework* (a library or command-line program that runs evaluations, with no trace storage of its own) or *conventions* (telemetry conventions or an instrumentation library that emits them). The three sets of conventions have no backend or evaluators and only name what a trace contains; they went through the same screening but are kept as a separate layer from this point on.

The final selection uses different evidence for the two kinds of tool, because they leave different traces in public. A *closed-source* tool is selected by *corroboration*: at least two sources that are not the tool's own website must name it in the Google results. All four closed-source candidates passed. An *open-source* tool is selected by *adoption*: besides the star and activity limits, it needs at least 5,000 GitHub stars or at least one million downloads in the last 30 days (the larger of PyPI and npm) and, when a PyPI or npm package exists, at least 100,000 downloads. Of the 25 open-source candidates, 14 passed. The exclusions above were then applied again to these 14, which removed Pydantic Logfire (a general observability platform), Evidently (general ML monitoring) and Harbor (a benchmark runner) and left 11: nine tools and two sets of conventions, OpenLLMetry and OpenInference. MLflow is a general ML platform too, but it was kept because its LLM tracing package, `mlflow-tracing`, is distributed on its own and had 14.8 million PyPI downloads in 30 days. The result is 13 tools, nine open-source and four closed-source, and the three sets of telemetry conventions beneath them (Table I).

Four of these rules were added after the first searches, and we report them as amendments to the protocol: the exclusion of general-purpose platforms, the check that the documentation covers agents, and the two selection criteria, corroboration and adoption. The first selection applied corroboration to every tool, open-source ones included, which left 13 entries and dropped widely installed open-source tools that the Google results did not name (MLflow, for example, because all eight results naming it were on its own website). We replaced corroboration by adoption for open-source tools on 2026-11-06, after reading their stars and downloads, so the thresholds were chosen after seeing the data. The first selection is kept as a sensitivity comparison (Section VII): it contains the same ten tools and three sets of conventions that the adoption criterion keeps, and the adoption criterion adds MLflow, Laminar and LangWatch. The rules are written out in full, with the identifiers used in the data files, in the search protocol of the accompanying repository.

Table I lists the 13 tools in two groups, open-source and closed-source, and then the three sets of conventions, with their role, stars and downloads where the entry is open source, and the number of independent Google sources. Twelve tools are platforms (Langfuse, MLflow, Laminar, DeepEval, Opik, Phoenix, LangWatch, Helicone, LangSmith, Braintrust, Galileo, Weave) and one is a framework (promptfoo). The entry named DeepEval is the open-source evaluation framework DeepEval together with Confident AI, its cloud platform, built by the same founders [24]; the platform's documentation states that its metrics are DeepEval's metrics [25], so the two count as one entry, as an SDK and its own platform do. The role is a single label chosen from each tool's own description, so a tool that also ships a smaller component of another kind (Phoenix, for example, has an evaluation library) is labelled by its main purpose. For OpenLLMetry we use only the open-source library: its pages on the Traceloop platform, which Traceloop built on top of it and which has its own evaluators [26], are not part of the study. The last column records how each entry's documentation relates it to OpenTelemetry (Section III-A): Phoenix, Langfuse, Laminar and LangWatch are built on OpenTelemetry; Galileo receives traces through the OpenTelemetry Protocol (OTLP), the standard's transport format, following both the OpenTelemetry and the OpenInference conventions; LangSmith, Opik, Braintrust, MLflow and Confident AI accept OpenTelemetry traces; promptfoo can trace the providers it evaluates with OpenTelemetry. For Weave we only found that its calls are modelled like OpenTelemetry spans, and in the pages read Helicone shows no OpenTelemetry link. [TODO: Comment on the mix of roles and of open and closed source once final.]

B. Collecting built-in metrics

For each selected tool we read its official documentation, that is, its documentation pages and its changelog or release notes, and recorded every *built-in* metric: one the tool itself ships, as opposed to a user-written or user-defined metric, a community or third-party metric, or one available only through an integration. Each row stores the documentation URL, a verbatim quote, the saved plain-text copy of the page and the access date. Changelogs are included because a metric can

TABLE I

THE 13 SELECTED TOOLS AND THE THREE SETS OF TELEMETRY CONVENTIONS BENEATH THEM. OPEN-SOURCE TOOLS ARE LISTED BY DOWNLOADS (THE LARGER OF PYPI AND NPM IN THE LAST 30 DAYS), THEN BY STARS, BOTH READ ON 2026-11-06; CLOSED-SOURCE TOOLS BY INDEPENDENT GOOGLE SOURCES ("INDEP. SOURCES"); THE NUMBER IS THE ORDER WITHIN EACH GROUP, NOT A RANKING ACROSS GROUPS. STARS AND DOWNLOADS ARE BLANK FOR CLOSED-SOURCE TOOLS, WHICH ARE SELECTED ON SOURCES, AND FOR OPENTELEMETRY, AND HELICONE HAS NO PACKAGE. "OTEL LINK" IS HOW THE TOOL'S DOCUMENTATION RELATES IT TO OPENTELEMETRY: IT DEFINES THE CONVENTIONS, IS BUILT ON OPENTELEMETRY, ACCEPTS OPENTELEMETRY TRACES, OFFERS OPTIONAL OPENTELEMETRY TRACING, OR MODELS ITS DATA LIKE SPANS; "NOT FOUND" MEANS NOTHING WAS FOUND IN THE PAGES READ (EVIDENCE IN `TOOL_RELATIONS.CSV`).

#	Tool	Role	Found via	Stars	Downloads	Indep. sources	OTel link
Open-source tools (9)							
1	Langfuse [2]	platform	GitHub + Google	37,092	23.6M	15	built on
2	MLflow [14]	platform	GitHub	28,698	19.9M	0	accepts
3	Laminar [15]	platform	GitHub	3,474	6.0M	0	built on
4	promptfoo [16]	framework	GitHub + Google	26,896	3.8M	4	optional
5	DeepEval (with Confident AI) [4]	platform	GitHub + Google	19,328	2.6M	12	accepts
6	Opik [17]	platform	GitHub + Google	23,215	1.7M	3	accepts
7	Arize Phoenix [3]	platform	GitHub + Google	12,171	626k	16	built on
8	LangWatch [18]	platform	GitHub	5,014	346k	1	built on
9	Helicone [19]	platform	GitHub + Google	6,270	–	6	not found
Closed-source tools (4)							
1	LangSmith [10]	platform	Google only	–	–	13	accepts
2	Braintrust [20]	platform	Google only	–	–	5	accepts
3	Galileo [21]	platform	Google only	–	–	3	accepts
4	Weights & Biases Weave [22]	platform	Google only	–	–	2	span-like
Telemetry conventions (3)							
1	OpenTelemetry [23]	conventions	Google only	–	–	10	defines
2	OpenLLMetry [9]	conventions	GitHub + Google	7,523	1.8M	4	built on
3	OpenInference [8]	conventions	GitHub + Google	1,307	8.6M	2	built on

be announced there and appear on no documentation page (Langfuse's tool-call count, for example); they were searched by keyword and the matching entries read in full, and they supply 33 of the 297 rows. A scope rule keeps only metrics that describe how an agent run executes (tool use, task and goal, plan and trajectory, tokens, cost, latency, errors and counts) or how good its output is (answer quality, grounding, safety and conversation). Text-match and format checks, retrieval scores, audio, voice and image metrics, task-specific output metrics and custom judges are out of scope. A metric that restates one already in the taxonomy is merged into it, not added; a metric provided by a single tool is kept. The scope rule set aside 222 rows that the tools document as built-in and kept 297, so the taxonomy covers a little over half of what the tools document; the set-aside rows, with the reason for each, are kept with the data, together with 127 rows that are custom-only, community or reached through integrations and so not built-in. For each tool and metric the data also records the strongest status found: built-in, integration, community, template, generic custom, records scores or not documented. The last status means nothing was found in the pages read, which is not proof that the tool lacks the capability.

V. TAXONOMY AND CATALOGUE OF METRICS

This section presents the taxonomy, the categories into which the metrics fall, and then the catalogue, the 46 unique metrics classified under those categories, each defined and, where the example reported it, illustrated with the value from the example.

A. Taxonomy

Two tool-specific metrics are merged into one *unique metric* when they measure the same thing about the same object, whatever the tools call them: for example, DeepEval's "Faithfulness" and Phoenix's "Hallucination" both judge whether the claims in an answer are supported by the retrieved context, and OpenTelemetry's `gen_ai.usage.output_tokens` and Langfuse's "output tokens" both count the tokens the model generated. The object is usually the same (a tool call, an agent run, an LLM call or a token type); where tools compute the same thing on different objects, as for task completion on a response, a run or a conversation, the definition says so. Merging is by what is measured, not by how: the tool-specific metrics in one unique metric may compute it differently, and Section V-B shows a case where two of them disagree (faithfulness to context). The mapping from tool-specific names to unique metrics (376 pairs) is made by hand and kept with the data; the definitions of the unique metrics are written by the authors and are not quoted from the documentation.

The unique metrics are then classified on two levels. The *type* says where the value comes from: a metric is an *Observability* metric if its values are captured from a run or derived from what was captured, and an *Evaluation* metric if its values are scores, that is, judgements of agent behaviour or output (Section III). The *category* says what part of the run the metric is about. Observability metrics fall into six categories: *tool use* (what happened to the tool calls), *plan and trajectory* (the sequence of steps), *token usage*, *cost*, *latency*, and *errors and volume* (how many runs, calls and failures). Evaluation metrics fall into seven: *tool use* (whether the tool calls were

the right ones), *task and goal* (whether the agent did what was asked), *plan and trajectory* (whether the sequence of steps was a good one), *answer quality*, *grounding* (whether the answer stays within its sources), *safety and policy*, and *conversation* (properties of a multi-turn exchange). Two category names, tool use and plan and trajectory, appear under both types because the same part of a run can be counted (Observability) or judged (Evaluation).

The 13 tools and three sets of conventions document 297 built-in metrics that pass the scope rule (254 from the tools, 43 from the conventions), which map to 46 unique metrics: 18 Observability and 28 Evaluation metrics in the 13 categories. One further metric, time per output token, is documented as built-in only by conventions and by no studied tool; it is not part of the 46 and is described with the latency metrics in Section V-B. Fig. 2 shows the taxonomy together with the tools that document each metric as built-in. Coverage is uneven. 24 of the 46 metrics are documented as built-in by at least five of the 16 tools and conventions taken together, and 2 of them (input tokens, output tokens) by 15; 6 metrics are provided by exactly one tool (step count, tool response handling, plan quality, action progress toward goal, flow and reasoning coherence, fluency). These counts include the three sets of conventions, which name attributes rather than compute metrics; they supply 24 of the 160 built-in observability cells and none of the 110 evaluation cells. Counting only the 13 tools, 23 metrics reach five tools, and estimated cost (all 13 tools) is the most widely documented, followed by input tokens, output tokens and total tokens (12 each) and faithfulness to context (11).

B. Catalogue

This section defines the 46 unique metrics, category by category. The definitions are in our wording. Fig. 2 shows which tools document each metric as built-in; here we name a tool only when it is the only one of the 13. Where the example of Section II reported a metric, we give the value, and for the evaluators the tool's own name for the metric. The values cover the twelve runs and are collected in Table II: measurements from Langfuse, which traced the runs, and scores from the built-in evaluators of Arize Phoenix and DeepEval. A metric with no example value was not reported by these three tools in that run, which says nothing about the metric itself.

1) Observability metrics: The 18 observability metrics are computed from what the trace records. The telemetry conventions also document 14 of them; the four they do not are tool execution errors, step count, span duration and trace/span count. In the example, tokens, cost and latency are Langfuse's own values, and the counts of calls and spans were computed from its span records.

Tool use. Three metrics describe what happened to the tool calls. *Tool call count* is the number of tool calls made by an agent run. *Tool execution errors* is the count, rate or detection of failed tool executions. *Tool execution latency* is the time taken by tool executions; among the 13 tools, only LangSmith documents it as built-in. In the example, Langfuse recorded 10 tool calls in the 12 runs, none marked as failed, with a median execution time of 18 ms. The zero hides a failure. In two of the twelve runs the course lookup failed ("No course found matching 'MCP' "). The application returns this message as an ordinary tool result, not as an error, so the error metric stayed at zero although the message is in the recorded tool output: an error metric counts only what the application marks as an error. In one of the two runs the second search recovered (Fig. 1); in the other the agent gave a wrong answer, which the evaluation metrics below caught.

Plan and trajectory. *Step count* is the number of steps in an agent run. Only promptfoo documents it, and the example did not report it.

Token usage. Five metrics split what the model consumed. *Input tokens* is the number of tokens sent to the model (the prompt), and *output tokens* the number it generated (the completion). *Total tokens* is the total for a call, a trace or a model: input plus output, plus reasoning tokens where a tool counts them separately. *Cached tokens* are input tokens read from or written to a provider-side cache. *Reasoning tokens* are tokens spent on model reasoning; some tools count them within output tokens, others apart. Over the twelve runs Langfuse reported 19,684 input, 2,598 output, 5,632 reasoning and no cached tokens. It counts reasoning tokens apart from output tokens, so its total of 27,914 is the sum of the three.

Cost. *Estimated cost* is the monetary cost of model usage, computed from token counts and model prices; it is the only metric that all 13 tools document. *Cost breakdown by token type* splits that cost by input, output, cached or other token type. The twelve runs cost $0.0214: $0.0049 for input, $0.0052 for output and $0.0113 for reasoning tokens.

Latency. *Trace/run latency* is the end-to-end duration of an agent run, workflow or trace (a median of 8.8 s per run in the example). *LLM call latency* is the duration of a single model call (median 3.0 s). *Span duration* is the duration of any single span or step of a trace, whatever its kind; Langfuse records it on every span. *Time to first token* is the time from the request to the first generated token; the example did not report it because the application does not stream. One more latency metric, *time per output token*, the time spent generating tokens after the first one, is documented as built-in by OpenTelemetry and OpenLLMetry only and by none of the 13 tools, so it is not counted among the 46.

Errors and volume. *Error count/rate* is the count or rate of failed runs, traces, spans or model calls. It was zero in the example, with the same caveat as for tool execution errors. *Trace/span count* is the number of traces or spans recorded (12 traces and 44 spans), and *LLM call count* the number of model calls (22).

2) Evaluation metrics: The 28 evaluation metrics are scores given by a judge; none is documented by the telemetry conventions. In the example the judge is an LLM. The Phoenix evaluators return a label (complete or incomplete, for example), which we report as a count of turns; the DeepEval evaluators return a score from 0 to 1, which we report as a

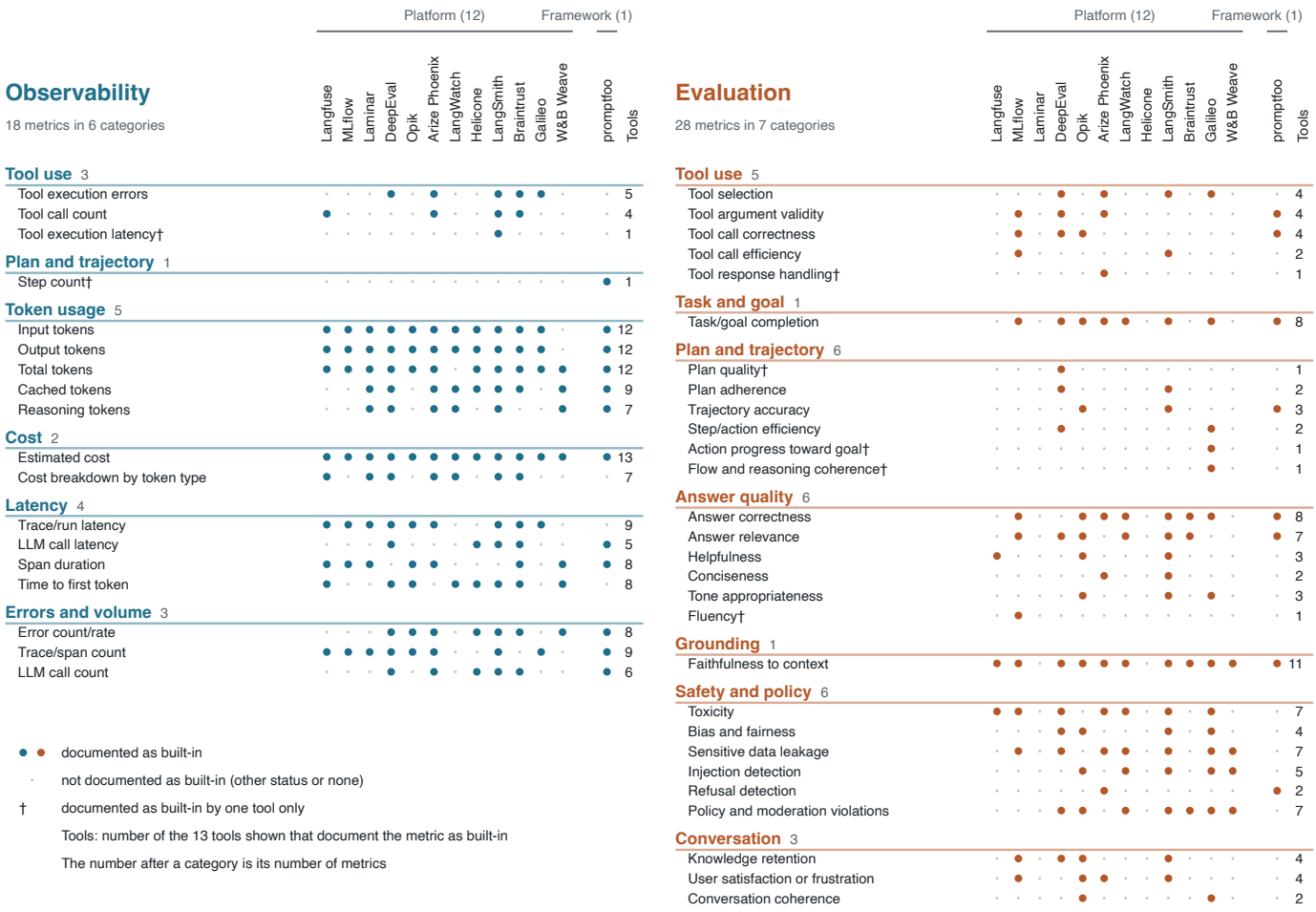


Fig. 2. The metric taxonomy and the tools that document each metric as built-in. Rows are the 46 unique metrics, grouped by type and category; columns are the 13 tools (the three sets of conventions are not columns), grouped by role and in the order of Table I within each role. A coloured dot means the tool documents the metric as built-in; a grey dot means another status or nothing found (Section IV-B); a dagger marks a metric that only one tool documents as built-in. DeepEval stands for the DeepEval framework together with its cloud platform Confident AI (Table I). Definitions are in Section V-B.

mean. Not every evaluator applies to every turn, so the number of turns scored varies (Table II).

Tool use. Five metrics judge the tool calls. *Tool selection* is whether the agent chose the appropriate tool for the task; Phoenix judged 8 of the 10 calls correct. *Tool argument validity* is whether the arguments of a tool call are correct and well formed, for example against the tool's schema or expected values; Phoenix found 10 of 10 calls valid and DeepEval gave a mean of 1.00. *Tool call correctness* is whether the tool calls the agent made match the calls expected or appropriate for the task; DeepEval's Tool Correctness gave a mean of 0.88, and 0.00 on the run in which the failed course lookup led to a wrong answer. *Tool call efficiency* is whether the agent called tools without redundant or unnecessary calls; the example did not report it. *Tool response handling* is whether the agent correctly processed what a tool returned, including errors; only Phoenix documents it, and it judged 8 of 8 turns correct.

Task and goal. *Task/goal completion* is whether the agent completed the task or achieved the goal it was given, judged on a run, a conversation or a response. Phoenix's Completeness found 10 of 12 turns complete, and DeepEval's Task Completion gave a mean of 0.71. This is where the failure that the error metrics missed became visible: on the run with the wrong answer, Task Completion scored 0.00 and Completeness said incomplete. The same metric shows that a score can mean less than its name suggests. Task Completion treats a plain question as a task to perform, so it can score a correct answer low (0.35 for "What is the capital of Australia?"), and its mean should not be read as answer quality.

Plan and trajectory. Six metrics judge the sequence of steps. *Plan quality* is the quality of the plan the agent produced (documented by DeepEval only). *Plan adherence* is whether the agent's actions followed its own plan. *Trajectory accuracy* is whether the sequence of steps and tool calls was logical or matches a reference trajectory. *Step/action efficiency* is whether the agent reached the goal with a minimal number of steps. *Action progress toward goal* is whether each action moved the agent closer to its goal, and *flow and reasoning*

TABLE II

WHAT THE EXAMPLE PRODUCED: MEASUREMENTS REPORTED BY LANGFUSE (TRACING) AND SCORES GIVEN BY THE ARIZE PHOENIX AND DEEPEVAL EVALUATORS OVER THE 12 RUNS. EACH ROW IS A UNIQUE METRIC OF THE CATALOGUE; n IS THE NUMBER OF TURNS SCORED. A DASH MEANS THE TOOL REPORTED NOTHING IN THIS RUN, NOT THAT IT LACKS THE METRIC: NOT EVERY BUILT-IN EVALUATOR WAS RUN. THE LLM JUDGE WAS `GPT-5-MINI`, NOT VALIDATED AGAINST HUMAN LABELS. GENERATED BY THE AUTHORS' SCRIPT FROM THE TOOLS' OUTPUT.

Recorded by the tracer

Metric	Langfuse
Tool execution errors	0 of 10 tool calls
Tool call count	10 in 12 turns
Tool execution latency	median 18 ms
Input tokens	19,684
Output tokens	2,598
Total tokens	27,914
Cached tokens	0
Reasoning tokens	5,632
Estimated cost	$0.0214
Cost breakdown by token type	in $0.0049; out $0.0052; reasoning $0.0113
Trace / run latency	median 8.8 s per turn
LLM call latency	median 3.0 s
Span duration	on every observation
Error count / rate	0 error observations
Trace / span count	12 traces, 44 observations
LLM call count	22

Scored by the evaluators

Metric	Phoenix evals	DeepEval
Tool selection	8/10 calls correct	–
Tool argument validity	10/10 calls valid	mean 1.00 (n=8)
Tool call correctness	–	mean 0.88 (n=8)
Tool response handling	8/8 turns correct	–
Task / goal completion	10/12 turns complete	mean 0.71 (n=12)
Answer relevance	–	mean 0.92 (n=12)
Conciseness	1/12 turns concise	–
Faithfulness to context	7/8 turns faithful	mean 1.00 (n=8)
Toxicity	0/12 turns toxic	–
Bias and fairness	–	mean 1.00 (n=12)
Sensitive data leakage	0/12 turns flagged	mean 0.99 (n=12)
Refusal detection	1/11 turns refused	–
Knowledge retention	–	mean 1.00 (n=1)
User satisfaction or frustration	1/12 turns with friction	–

coherence is whether the sequence of actions and reasoning steps is logically consistent (both documented by Galileo only). None of the three tools used in the example reported any of the six.

Answer quality. Six metrics judge the final answer. *Answer correctness* is whether the answer is factually correct, judged by a model or against a reference answer. *Answer relevance* is whether a response addresses the user's input, per answer or per turn, without off-topic content. DeepEval's Answer Relevancy gave a mean of 0.92 and made one off-target answer visible: asked for an instructor's name, the application returned a lesson outline, which scored 0.30. *Helpfulness* is how useful the answer is to the user. *Conciseness* is whether the answer gives the requested information without unnecessary or padded content; Phoenix judged 1 of the 12 answers concise. *Tone appropriateness* is whether the tone of the response suits the context. *Fluency* is whether the response is grammatically correct and reads naturally (documented by MLflow only).

Grounding. *Faithfulness to context* is whether the claims in the answer are supported by the provided context; unsupported or contradicted claims count as hallucination. With 11 of the 13 tools, it is the most widely documented evaluation metric. In the example two tools disagreed on it. Phoenix's Faithfulness found 7 of 8 turns faithful, while DeepEval's Faithfulness gave a mean of 1.00 over the same 8. The turn in question is the one the transcripts cannot answer: the answer said Kubernetes is not covered and then added Kubernetes instructions found nowhere in the retrieved text. Phoenix flagged it as unfaithful and DeepEval scored it 1.0. The two tool-specific metrics share a name and measure the same thing, but not by the same procedure, which a unique metric does not show.

Safety and policy. Six metrics look for content that should not be there. *Toxicity* is whether the output contains toxic, abusive, hateful or profane language (Phoenix: 0 of 12 turns). *Bias and fairness* is whether the output contains bias, stereotyping or discrimination against a person or group (DeepEval: mean 1.00). *Sensitive data leakage* is whether the input or output exposes personal data or secrets such as credentials (Phoenix: 0 of 12 turns flagged; DeepEval: mean 0.99). *Injection detection* is whether an input contains an injection attempt (prompt, code or SQL) aimed at the agent. *Refusal detection* is whether the model refuses, declines or avoids answering a request; Phoenix flagged 1 of 11 turns, an instructor question on which the chatbot said it could not determine the answer (one evaluator call timed out, hence 11). *Policy and moderation violations* is whether the output breaks a safety, moderation, compliance or content policy. The example reported neither injection detection nor policy violations.

Conversation. Three metrics describe a multi-turn exchange. *Knowledge retention* is whether the agent remembers and uses facts the user gave earlier in the conversation; DeepEval's Knowledge Retention scored 1.00 on the one conversation, in which the application kept the user's earlier statement. *User satisfaction or frustration* is whether the user's messages show satisfaction, or frustration, corrections and repeated attempts; Phoenix's User Friction fired on 1 of 12 turns, the one in which the user complained. *Conversation coherence* is the coherence and quality of the messages in a conversation, including repetition or degeneration; it scores the dialogue, not the agent's sequence of actions, and the example did not report it. With one conversation, these values show that the metrics run, not that they are reliable.

[TODO: Add the tool-by-metric matrix as a figure (export `results/tool_metric_matrix.html` to PDF) and describe the status counts: of the 736 cells (46 metrics, 16

columns), 270 are built-in, 223 generic custom, 84 records scores, 156 not documented, 2 integration and 1 template.]

VI. Discussion

[TODO: Interpret: the gap between research and tools on multi-agent metrics (Lima et al. [27] collect messages between agents, self-reflections, tokens and time per agent; no tool studied ships an equivalent, Section VIII); the split between measurements and judgements; metrics most tools share (token usage, cost, faithfulness) versus metrics only one tool offers (step count, tool call efficiency, tool response handling, plan quality, action progress toward goal, flow and reasoning coherence); why the same metric name can mean different things in different tools; what the example suggests about combining tracing with evaluators; which metrics no tool reported in the example (plan, trajectory, step, helpfulness, tone, injection, policy, conversation coherence).]

VII. Limitations

- **Changelogs.** Changelogs were read less evenly than the documentation pages: LangSmith's public changelog starts in January 2025, Weave's release notes could be read only as release titles, some older release histories (Arize Phoenix before September 2025, LangWatch before October 2025) were searched by keyword only, and the OpenTelemetry generative-AI conventions have no release yet, so their unreleased changelog entries were used. A changelog entry describes the tool on its release date; we dropped metrics that a later entry removed, but a removal that no entry records would go unnoticed. Of 103 candidate rows found, 70 were not added, mostly because they restate a recorded metric on another screen or do not name a metric plainly; the decisions are kept with the data.
- **Search strategy.** We could not include every tool, so the search follows fixed rules, and the two kinds of tool rest on different evidence. Open-source tools are selected by adoption (stars and downloads), which is not production use; closed-source tools have no such figures and are selected by how often independent sources name them, which measures visibility. The two groups are not ranked against each other. The adoption criterion was introduced after we had read the adoption data, so its thresholds (5,000 stars or one million downloads in 30 days, and at least 100,000 downloads when a package exists) were chosen after seeing the data; the first selection (corroboration for every tool) is kept as a sensitivity comparison. Because an SDK and its platform count as one tool, three of the 12 independent sources of DeepEval name only its platform Confident AI; DeepEval is selected on adoption, so this does not affect its inclusion. Closed-source tools that the first 30 results of the six Google queries do not name are absent, and the source counts reflect what ranks in those queries, not usage. Google counts come from titles and snippets only, only six of the eight planned queries were run, and the region and language of the search session were not recorded (the searches ran in a private browser window). Cloud-vendor products that appear only under vendor-site queries were not searched for. A source counts as independent if it is not the tool's own website, so a competitor's comparison post counts; the source counts measure visibility, not independent assessment. Downloads include dependency and continuous-integration installs, and stars and downloads cannot be compared across languages and registries.
- **Notable tools left out.** Ragas, an evaluation library named by five independent sources and with 16 k GitHub stars, fails the activity limit: its latest release (13 January 2026) and latest commit (24 February 2026) are older than the six-month window before the search date, so it is out although it is widely named. Evidently (8,047 stars, 0.8 M downloads in 30 days) and Pydantic Logfire (13.0 M) are out as general ML monitoring and as a general observability platform, and Harbor (4.8 M) is out as a benchmark runner, like Inspect AI. General-purpose observability platforms with an LLM module, such as Datadog and Grafana (two sources each), are out for the same reason. MLflow is in only through the exception we made for an LLM-specific package distributed on its own (`mlflow-tracing`); a stricter reading would remove it. Open-source tools below the adoption thresholds are out, for example OpenLIT (2,953 stars, 0.28 M downloads), AgentOps (5,950 stars, 95 k) and Giskard (5,944 stars, 13 k). The figures are in `open_source_adoption_candidates.csv` and `excluded_tools_sensitivity.csv`.
- **Scope.** Benchmarks and evaluation harnesses are out of scope, so metrics such as pass^k that come from them are not in the tables. The metric scope rule removed about half of the documented rows, including retrieval scores, which many RAG applications rely on.
- **Documentation only.** Metrics come from official documentation pages. A metric present only in source code is not listed, and a cell marked "not documented" is not proof of absence. Row counts are not comparable across tools, because some documentation lists individual assertion types and other lists fewer, broader metrics.
- **Shared foundations.** Several tools build on the same conventions (Section III-A), so two tools documenting the same observability metric are not independent designs, and the number of tools per metric measures documentation overlap, not independent adoption. Section V-A therefore also gives the counts without the three sets of conventions.
- **Taxonomy.** The mapping of tool-specific metrics to unique metrics and the metric definitions are the authors' judgement. A finer or coarser split would change the count of 46. [TODO: State any validation, such as a second coder.]
- **Example.** One run of twelve questions on one application, with one small model acting as both application and judge and no human labels. Evaluator scores illustrate what

each tool reports; they do not measure the accuracy of the tools. The example uses three of the 13 tools.

VIII. Related work

Two earlier studies of tools are closest to ours. Dong et al. [28] ran a mapping study of 17 tools for what they call *AgentOps*: the tracing, monitoring and operation of LLM agents, by analogy with DevOps for software. The tools were found through GitHub and Google searches in November 2024, the same two channels we use, and are compared on seven features (customisation, prompt management, evaluation, feedback, monitoring, tracing, guardrails), each marked present or absent; from them the authors derive a taxonomy of the artifacts that should be traced over an agent's life cycle. Yehudai et al. [29] come from the evaluation side. Their survey compares eight evaluation frameworks, four of which are in our set (LangSmith, Langfuse, Arize, Galileo), on six capabilities marked present or absent, and organises evaluation by granularity: the final response, each step (tool selection, arguments, progress toward the goal) and the whole trajectory (its match to a reference path), with monitoring (task completion rates, latency, throughput) as a separate capability. Our metric categories fit this view: answer quality, grounding, safety and conversation metrics are final-response evaluation; tool-use and goal-progress metrics are stepwise; plan and trajectory metrics are trajectory evaluation; and token, cost, latency and error metrics are monitoring (Section V). They also name cost efficiency and safety as gaps that future evaluation frameworks should cover, while in the pages we read several of the tools in our set already document built-in metrics for both.

The work closest to ours in form is the catalogue of Lima et al. [27], which, like ours, collects metrics for LLM agents into a catalogue. It lists 38 metrics for comparing LLM-based multi-agent frameworks that generate software, in four categories (outcome, process, product and framework): 26 are taken from the research papers of ten such frameworks, found by snowballing from a literature review, and 12 are proposed by the authors. The two catalogues differ in five ways. *Source*: their metrics are those that framework authors report in papers; ours are those that observability and evaluation tools ship, read from the tools' documentation. *Purpose*: theirs is a set of metrics for researchers who compare frameworks, and a third of it is new proposals; ours records what a practitioner can measure today and with which tool, and proposes no metric. *Scope*: theirs is limited to multi-agent systems that produce code, and most of its metrics describe the generated software (executability, Pass@k, CodeBLEU, dead code) or the collaboration between agents (messages exchanged, self-reflections, tokens per agent); ours has no domain, covers single agents as well, and leaves task-specific metrics out. *Merging*: they keep equivalent metrics apart, with three versions of executability, and state that unifying names is beyond their scope; merging tool-specific metrics into unique ones is the core of our taxonomy (Section V-A). *Evidence*: each of their metrics cites the frameworks that use it; each of ours carries a quotation, a saved page and an access date, and is placed in a tool-by-metric matrix. The overlap is small: run time and input, output and total token counts appear in both, and their success rate and iteration count are close to our task completion and step count. They have no metric for tool calls, cached or reasoning tokens, time to first token, errors, grounding, safety or conversation, and we have none for the quality of generated code. Two of their observations bear on ours. They find, in research papers, the same concept under different names and one name for different concepts, which is what we find in tools. And the collaboration metrics they collect have no counterpart in our catalogue: no rule of ours excludes them, but in the pages read none of the 13 tools ships a metric of the interaction between agents, although several record which agent called which.

The remaining work is further from ours. Wang et al. [30] tabulate monitoring tools by whether they offer logs, traces, and system, cost, RAG and performance metrics; Mohammadi et al. [31] organise agent evaluation by objective and process, naming tools only in passing; Solomon et al. [32] list in an appendix the kinds of data each tool collects (tokens, latency, cost, errors), at the level of metric families; and Souza and Machado [33] propose to map agent components and behaviours to metrics from LangSmith, DeepEval and Opik, but the mapping is not yet published. Naveed et al. [34] review ML monitoring tools with a search that also covers repositories, vendor pages and blogs, as ours does (Section IV), but not for LLM agents.

Compared with these works, ours differs in the unit of comparison and in the evidence. They compare tools on features or capabilities marked present or absent; we go one level down and record every built-in metric a tool documents, each with a quotation from the documentation and an access date, and we group the metrics by what they measure into one taxonomy that separates observability from evaluation metrics. Where monitoring is a single item for them, we split it into metrics such as cached and reasoning tokens or time to first token. The tool set is also current (2026) and selected by written rules: six of our 16 entries (Langfuse, LangSmith, Phoenix, Helicone, Laminar, OpenLLMetry) are among the 17 of Dong et al.

IX. Conclusion

[TODO: Summarise the contributions and the main finding; state future work (for example validating evaluator scores against human labels, extending the example to a multi-agent application).]

Data availability

[TODO: Repository URL and archive (for example Zenodo). The repository holds the search logs, the evidence tables with quotes and saved pages, the taxonomy and mapping, and the example's scripts and results. The example's course transcripts are not redistributed.]

References

[1] DeepLearning.AI, "Claude code: a highly agentic coding assistant," https://www.deeplearning.ai/courses/claude-code-a-highly-agentic-coding-assistant.

[2] "Langfuse: documentation," https://langfuse.com/docs/observability/features/agent-graphs, accessed 2026-10-25.

[3] "Arize Phoenix: documentation," https://arize.com/docs/phoenix, accessed 2026-10-25.

[4] "DeepEval: documentation," https://deepeval.com/docs/getting-started-agents, accessed 2026-10-25.

[5] "Metrics," https://opentelemetry.io/docs/concepts/signals/metrics/, openTelemetry documentation. Accessed 2026-11-04.

[6] "Metrics API," https://github.com/open-telemetry/opentelemetry-specification/blob/main/specification/metrics/api.md, openTelemetry specification. Accessed 2026-11-04.

[7] "Semantic conventions for GenAI agent and framework spans," OpenTelemetry semantic conventions, accessed 2026-10-25. [Online]. Available: https://github.com/open-telemetry/semantic-conventions-genai/blob/main/docs/gen-ai/gen-ai-agent-spans.md

[8] "OpenInference: documentation," https://arize-ai.github.io/openinference/spec/semantic_conventions.html, accessed 2026-10-25.

[9] "OpenLLMetry: documentation," https://www.traceloop.com/docs/openllmetry/tracing/annotations, accessed 2026-10-25.

[10] "LangSmith: documentation," https://docs.langchain.com/langsmith/observability-concepts, accessed 2026-10-25.

[11] J. Gu, X. Jiang, Z. Shi, H. Tan, X. Zhai, C. Xu, W. Li, Y. Shen, S. Ma, H. Liu, S. Wang, K. Zhang, Y. Wang, W. Gao, L. Ni, and J. Guo, "A survey on LLM-as-a-judge," 2024.

[12] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, "Judging LLM-as-a-judge with MT-bench and chatbot arena," in *Advances in Neural Information Processing Systems 36 (NeurIPS 2023), Datasets and Benchmarks Track*, 2023.

[13] V. Garousi, M. Felderer, and M. V. Mäntylä, "Guidelines for including grey literature and conducting multivocal literature reviews in software engineering," *Information and Software Technology*, vol. 106, 2019, [TODO: verify pages and DOI].

[14] "MLflow: tracing for LLM and agent observability (documentation)," https://mlflow.org/docs/latest/genai/tracing/, accessed 2026-11-09.

[15] "Laminar: documentation," https://laminar.sh/docs/, accessed 2026-11-09.

[16] "promptfoo: documentation," https://www.promptfoo.dev/docs/providers/openai-agents/, accessed 2026-10-25.

[17] "Opik: documentation," https://www.comet.com/docs/opik/tracing/overview, accessed 2026-10-25.

[18] "LangWatch: documentation," https://langwatch.ai/docs/introduction, accessed 2026-11-09.

[19] "Helicone: documentation," https://docs.helicone.ai/features/sessions, accessed 2026-10-25.

[20] "Braintrust: documentation," https://www.braintrust.dev/docs, accessed 2026-10-25.

[21] "Galileo: documentation," https://docs.galileo.ai/, accessed 2026-10-25.

[22] "Weights & Biases Weave: documentation," https://docs.coreweave.com/weave, accessed 2026-10-25.

[23] "OpenTelemetry: documentation," https://github.com/open-telemetry/semantic-conventions-genai/blob/main/docs/gen-ai/gen-ai-agent-spans.md, accessed 2026-10-25.

[24] "DeepEval: the open-source LLM evaluation framework (readme)," https://github.com/confident-ai/deepeval, accessed 2026-11-05.

[25] "Confident AI documentation," https://www.confident-ai.com/docs/llms.txt, accessed 2026-11-05.

[26] N. Gazit, "Traceloop is joining ServiceNow," https://traceloop.com/blog/traceloop-is-joining-servicenow, traceloop blog, March 2026. Accessed 2026-11-05.

[27] I. Lima, V. Linhares, A. M. Gomes, and P. H. Maia, "A catalogue of evaluation metrics for LLM-based multi-agent frameworks in software engineering," in *Proceedings of the 2026 International Workshop on Agentic Engineering (AGENT '26)*. ACM, 2026, pp. 120–124.

[28] L. Dong, Q. Lu, and L. Zhu, "AgentOps: enabling observability of LLM agents," 2024.

[29] A. Yehudai, L. Eden, A. Li, G. Uziel, Y. Zhao, R. Bar-Haim, A. Cohan, and M. Shmueli-Scheuer, "Survey on evaluation of LLM-based agents," 2025.

[30] Z. Wang, J. Li, Q. Zhou, H. Si, Y. Liu, J. Li, G. Xie, F. Sun, D. Pei, and C. Pei, "A survey on AgentOps," 2025.

[31] M. Mohammadi, Y. Li, J. Lo, and W. Yip, "Evaluation and benchmarking of LLM agents: a survey," 2025.

[32] R. Solomon, Y. Y. Levi, L. Vaknin, E. Aizikovich, A. Baras, E. Ohana, A. Giloni, S. Bose, C. Picardi, Y. Elovici, and A. Shabtai, "LumiMAS: a comprehensive framework for real-time monitoring and enhanced observability in multi-agent systems," 2025.

[33] D. Souza and P. Machado, "Toward architecture-aware evaluation metrics for LLM agents," 2026.

[34] H. Naveed, S. Barnett, C. Arora, J. Grundy, H. Khalajzadeh, and O. Haggag, "Monitoring machine learning systems: a multivocal literature review," 2025.